

Intel® performance hybrid architecture & software optimizations

Part Two: Developing for Intel performance hybrid architecture

This document is the second in a series of whitepapers that will serve to provide an overview of the Intel's performance hybrid architecture.

Authors

Nikhil Rukmabhatla

Software Enabling and
Optimization Engineer

Rajshree Chabukswar

Senior Principal Engineer

Sneha Gohad

Software Enabling and
Optimization Engineer

Michael Chynoweth

Intel Fellow

Introduction

This second whitepaper deals with considerations from developing for performance hybrid architecture and software optimizations. In developing for Intel's performance hybrid architecture, certain considerations need to be kept in mind. While most of these considerations are software-based, a few deals with the actual hardware architecture. Several of these considerations are covered in this paper.

Best Known Practices

To develop applications using Intel's performance hybrid architecture, we recommend developers follow these best-known practices, each elaborated in the following section:

- Stay current on updates to operating systems and optimized libraries.
- Use the Windows Power Throttling information embedded in the Windows API and Microsoft QoS API to give Quality of Service (QoS) hints to the scheduler. These hints improve both performance and energy efficiency on threads and processes.
- Help ensure optimal performance under power-constrained settings by not setting hard affinities on either threads or processes (e.g. APIs like `SetThreadAffinityMask()`). This allows the Operating System (OS) to provide the optimal core selection for Intel's performance hybrid architecture and using feedback from the Intel Thread Director.
- Utilize available scheduling opportunities, fully enabled through the Intel Thread Director, to assist in development and performance.
- Replace active busy spin-waits with lightweight waits, ideally using the new **UMWAIT/TPAUSE** instructions which save more energy and gives developer more control over wait duration. Traditional **PAUSE** instructions have fixed latency of 140 cycles.
- Use software libraries that are already optimized for performance hybrid architecture.

For workloads and configurations visit www.Intel.com/PerformanceIndex. Results may vary.

Table of Contents

Introduction	1
Best Known Practices.....	2
Key Points	2
Identifying Key Systems	2
Microsoft Windows APIs	2
Windows QoS APIs	3
Ensuring Optimal Performance.....	3
Scheduling Opportunities.....	4
Optimizations.....	5
Active Spins.....	5
12th Generation Support	6
Optimized Libraries	6
Conclusion.....	7
References.....	7

KEY POINTS

Identifying Key Systems for performance hybrid architecture

When working within performance hybrid architecture, it is necessary to identify and to understand key systems like the topology and the OS to be sure the performance hybrid architecture is working properly. The topology in your system can validate whether or not scheduling is functioning normally. One of the key ways of identifying topology is through a processor's CPUID. This is why, before getting started, we recommend that you find and document each processor's CPUID. The CPUID can be found by downloading and running our Intel Processor Identification Utility, which will report the CPUID information for the processor being tested. The CPUID is a combination of the processor family, processor model, and processor stepping reported in a hexadecimal format (for example, 806E9 represents an Intel Core™ i5-7300u @ 2.60GHz, also known as Kaby Lake).

Similarly, different OSes have different needs, values, languages, and even versions. This makes optimizing for each OS complicated. However, there are different tools out there to detect and diagnose your OS in case you need it. We recommend using Version Helper API for this diagnosis. You can read more about Version Helper API [here](#) or go to the docs.microsoft.com site and search for the keyword “Version Helper”.

Microsoft Windows APIs

The Microsoft Windows API (or Win32 API) continues to be a great application resource. An application developed using Windows API can be run successfully on all versions of Windows. We recommend using Windows API because, when used correctly, the Windows API can provide information that helps ensure your system is getting the full benefit of the performance hybrid architecture.

Sysinfoapi.h provides many useful APIs like `GetSystemInfo`, `GetLogicalProcessorInformation` (GLPI) and `GetLogicalProcessorInformation` (GLPIEX). With the right usage, developers can access system information through these APIs, including topological relationships between underlying hardware.

For workloads and configurations visit www.Intel.com/PerformanceIndex. Results may vary.

GLPI returns a **SYSTEM_LOGICAL_PROCESSOR_INFORMATION** structure that displays the Relationship and ProcessorMask fields. A processor mask identifies the processors in the structure while the relationship identifies whether or not the processors share anything. This can help tremendously in understanding workloads and scheduling.

Meanwhile, **GLPIEX** is a more recent iteration of GLPI and has more information through its similar **SYSTEM_LOGICAL_PROCESSOR_INFORMATIONEX**. Unlike GLPI and system logical processor information that is not EX, the **GLPIEX** has processor group as well. Processor groups only become necessary when dealing with systems that are beyond 64 logical processors.

Additionally, For developing with performance hybrid architecture, we recommend using the API `GetSystemCPUSetInformation()` from [Processthreadsapi.h](#) to query the available CPU Sets on the system and their current state, allowing developers to know their targeted E-cores and P-cores.

The following pseudo-code demonstrates how to compare bit masks from **GLPI** return value in C++.

[illegible]

Figure 1. Code snippet showing mask comparison

```

        // Physical processor is related to l1 cache mask
    }
    else
    {
        // Physical processor is not related to the L1 Cache
    }

    if ((physical_processor_mask & l1_cache_mask) &&
        (logical_processor_mask & l1_cache_mask))
    {
        // Physical processor and logical processor share L1 cache.
    }
    else
    {
        // Physical processor and logical processor do not share an L1 cache.
    }
}

```

Figure 1 (cont.). Code snippet showing mask comparison

Microsoft QoS APIs

We recommend setting the QoS low to indicate that work not critical to application performance (e.g. garbage collections, IO not related to performance etc). We recommend using Microsoft's QoS API to set the QoS low. Along with setting priority assignments, doing this will help you ensure that critical threads get the most P-cores and non-critical work (indicated by the QoS API) can go to E-cores. Microsoft provides two APIs to manage work at a process and thread-level, depending on its relative importance:

- `SetProcessInformation()`
- `SetThreadInformation()`

```

//How to code: let system manage all power throttling

// process level
PROCESS_POWER_THROTTLING_STATE powerThrottlingProcessState = {
    PROCESS_POWER_THROTTLING_CURRENT_VERSION,
    0, // ControlMask to allow caller control of the power throttling
    0 // StateMask to manage QoS setting
};
SetProcessInformation(GetCurrentProcess(), ProcessPowerThrottling, &powerThrottlingProcessState, sizeof(PROCESS_POWER_THROTTLING_STATE));

// or thread level
THREAD_POWER_THROTTLING_STATE powerThrottlingThreadState = {
    THREAD_POWER_THROTTLING_CURRENT_VERSION,
    0, // ControlMask to allow caller control of the power throttling
    0 // StateMask to manage QoS setting
};
SetThreadInformation(GetCurrentThread(), ThreadPowerThrottling, &powerThrottlingThreadState, sizeof(THREAD_POWER_THROTTLING_STATE));

// How to code: Eco QoS for improved efficiency and battery life

// process level
PROCESS_POWER_THROTTLING_STATE powerThrottlingProcessState = {
    PROCESS_POWER_THROTTLING_CURRENT_VERSION,
    PROCESS_POWER_THROTTLING_EXECUTION_SPEED, // ControlMask to allow caller control of the power throttling
    PROCESS_POWER_THROTTLING_EXECUTION_SPEED // StateMask to manage QoS setting
};
SetProcessInformation(GetCurrentProcess(), ProcessPowerThrottling, &powerThrottlingProcessState, sizeof(PROCESS_POWER_THROTTLING_STATE));

// or thread level
THREAD_POWER_THROTTLING_STATE powerThrottlingThreadState = {
    THREAD_POWER_THROTTLING_CURRENT_VERSION,
    THREAD_POWER_THROTTLING_EXECUTION_SPEED, // ControlMask to allow caller control of the power throttling
    THREAD_POWER_THROTTLING_EXECUTION_SPEED // StateMask to manage QoS setting
};
SetThreadInformation(GetCurrentThread(), ThreadPowerThrottling, &powerThrottlingThreadState, sizeof(THREAD_POWER_THROTTLING_STATE));

```

Figure 2. Code snippet showing power throttling

Ensuring Optimal Performance Under Power-Constrained Scenarios

We discourage hard affinities of a thread or a process to a CPU core since it can impede an OS from making the right decisions on where a task is best scheduled and prevent it from efficiently load balancing.

We also recommend not having software attempt to determine which Instruction Set Architecture (ISA) will run on specific cores, even though there may be performance differences among various ISAs running on E-cores and P-cores. Instead, we recommend letting the Intel Thread Director provide feedback to the OS with the current run-time status of each core and the current power and performance needs of application threads so that OS can choose the right cores for right ISA.

Developers may not be aware of any run time power/thermal constraints that end user might see in the field. Avoiding affinity ensures OS and Intel Thread Director makes the optimal decision based on current context of the system.

HINT: You can determine whether or not affinities have been applied in application by searching the source code with API calls, specifically – 'SetThreadAffinityMask()' and 'SetProcessAffinityMask()'. Developer simply needs to remove usage of these API calls from their source to remove usage of hard affinities.

Below is a sample call of the `SetProcessAffinityMask` which sets a hard affinity and should be removed:

```
//Set the processor affinity mask
if (!SetProcessAffinityMask(processInfo[i].hProcess, affinityMask)) {
    printf("SetProcessAffinityMask failed, mask = %16llx \n", affinityMask);
}
```

Figure 3. Code snippet showing `SetProcessAffinityMask`

Scheduling Opportunities

Our 12th Generation Intel Core processors introduce new scheduling techniques, which are fully enabled through the Intel Thread Director and the Operating System, to be used when running a workload on performance hybrid architecture. These techniques will help in achieving maximum performance and power benefits. The below sequence explains a typical scheduling scenario:

- i. Performant P-cores are used first for single threaded (ST) and multi-threaded (MT) performance.
- ii. Workloads with higher parallelization utilize the efficient E-cores for further scalable performance.
- iii. HW siblings threads running different logical processors of the same physical core on P-cores are used last to avoid any contention impacting performance when all P-cores and E-cores are busy.
- iv. Background work runs on E-cores to reduce impact on MT performance.
- v. Less performant threads are moved to efficient E-cores to make more headroom for more performant threads per Intel Thread Director feedback.
- vi. Intel Thread Director feedback is utilized to leverage performance differences in threads, choose the right core for the right thread, and minimize the use of too many thread context-switches at the same time.
- vii. Efficient E-cores are leveraged to conserve power.
- viii. Threads are characterized to better leverage core performance and efficiency differences.
- ix. Cores are leveraged via Power Throttling Quality of Service APIs.

Since the 12th Generation Intel Core processors are Intel's first performance hybrid architecture with performant P-cores and more efficient E-cores, the implications of having an unbalanced (disparate) thread execution had to be considered carefully. While performance hybrid architecture shows promising average speed-up in multi-threading scenarios, there can be corner cases where more performance can be achieved helping ensure the right thread is scheduled on the right core.

For this reason, we collaborated with OS vendors and tools/libraries developers to:

- Refine OS scheduling with optimizations and inclusion of the Intel Thread Director for scheduling decisions.
- Prepare threading libraries to ensure that threads are optimally scheduled.

Windows 11 has already incorporated Intel Thread Director for use when mapping a thread to an optimal core for performance. Further OS scheduling enhancements have been incorporated to help differentiate the QoS of various threads and guide scheduling. This differentiation is managed based on whether an application has foreground or background status, and whether it impacts the user experience of the customer. The goal was to prepare OS, libraries, and frameworks for optimal thread scheduling for best performance and throughput, reducing the enabling burden for application developers.

As we go forward, we are continually building our knowledge base around commonly expected issues, tools, and techniques for optimizations. These one-time optimizations and enablements will carry forward to future Intel client roadmap as well. ISVs are encouraged to set their own QoS to better inform the OS on which threads are non-critical to application performance and best kept in the background.

Understanding Intel Thread Director Optimizations

Intel Thread Director was built on top of previously existing Lakefield Hardware Guided Scheduling (HGS) support and understanding how it works deserves some individual consideration. As mentioned earlier, the OS scheduler allocates the best core for a software thread depending on its runtime properties. The Intel Thread Director provides hints to the OS scheduler with thread-related information. Using these hints, the OS can choose between energy efficiency (EE) and performance (PERF) depending on system parameters like power policy, battery slider, etc. This performance and efficiency management depends greatly on the software profile and microarchitecture capabilities.

For workloads and configurations visit www.intel.com/PerformanceIndex. Results may vary.

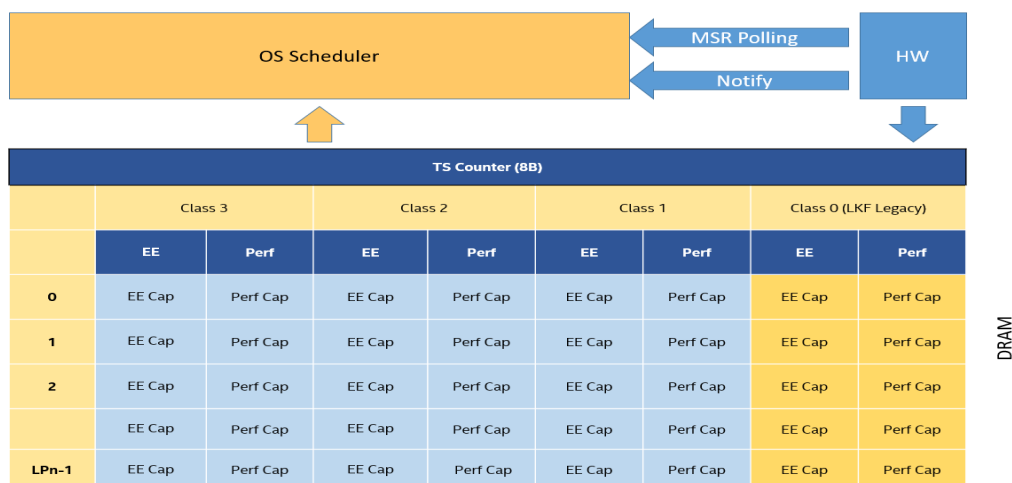


Figure 4. Intel Thread Director determines and displays capabilities and the way that information is communicated to OS

Per Class ID,

- ◇ PERF capabilities provide relative performance level of the larger and efficient logical processor (higher values indicate the higher performance). OS sorts threads by core-to-core performance ratio and schedules by core performance order.
- ◇ EE capability provides relative energy efficiency level of a logical processor (higher EE values indicate higher energy efficiency requirement for thread).

The OS then decides on which class to assign each application. The various classes indicate performance difference between the P-cores and E-cores:

- Class 0 indicates operations that are similarly performant on P-cores and E-cores.
- Class 1 indicates ISAs (like AVX2-FP32) where P-cores are more performant than E-cores.
- Class 2 indicates emerging ISAs (such as AI) where P-cores are more performant than E-cores.
- Class 3 indicates spin-waits and work that doesn't scale with higher performance.

Note: Classes were also introduced to track lightweight waits like **TPAUSE/UMWAIT** so that the OS prioritizes threads with a high amount of time in busy lightweight waits to E-cores for enhanced power gains.

Active Spins in Applications

Spinning (also known as busy waits) is a pattern when a process or a thread recurrently checks for a condition while spinning on a core, taking up compute resources. Active spins are present in many applications and benchmarks, so it was critical having a methodology to detect performance hybrid architecture and potentially move excessive spinning to the E-core.

The spin time increases as we scale cores with performance hybrid architecture due to the threading libraries spinning in between 'parallel_for(s)' and other parallel constructs. These active spins tend to occur in the following situations:

- Waiting on a hardware resource
- Producer/consumer model when one thread is waiting on another
- Backing off a lock due to high contention
- Thread imbalance in thread pools after threads in the pool performed with parallel work
- In between parallel work many threading libraries such as OpenMP will spin in anticipation of another parallel region.

Generally, applications spin attempting to give up their quanta to any other ready thread by spinning on APIs like **'SwitchToThread()'** and **'Sleep(0)'**. However, if no other dedicated threads are ready, then the spinning threads will consume valuable CPU resources and power. From both a power and a performance perspective, the recommendation is to replace these active spins with **'Userland MWait'** (UMWAIT) or **'Timed Pause'** (TPAUSE) instructions.

The instructions **UMWAIT** and **TPAUSE** enter a lightweight yet responsive C-State (C0.1 or C0.2), saving power and not requiring a spin on the 140 core-cycle **PAUSE** instruction on client. All three waits including time in **UMWAIT**, **TPAUSE** and **PAUSE** are all detected by Intel Thread Director and excessive time is tagged as a potential busy spin.

Thread pools that are done with parallelism—such as closing out a `parallel_for()`—will often spin, anticipating another parallel construct. This keeps the cores from idling.

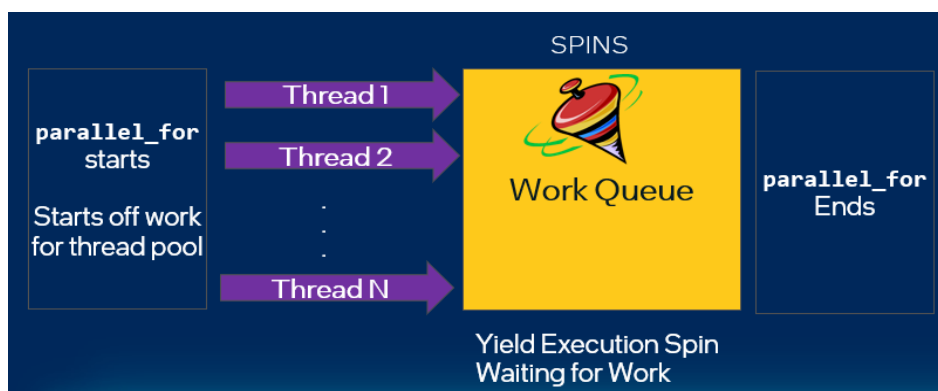


Figure 5. Threads spin in parallel constructs during load imbalance and at the end of the construct waiting for more work

We recommend utilizing **UMWAIT** or **TPAUSE** because that will save power over an active spin as well as over a spin on a **PAUSE** instructions.

12th Generation Intel Core processors ISA Support

With our 12th Generation Intel Core processors, we are supporting common subsets of ISA for both P-cores and E-cores. To make the 12th Generation Intel Core processor ISA client relevant, we removed AVX512 support from the 12th Generation Intel Core processor P-cores, opting instead to backport AVX512-VNNI features to AVX. By doing that, we now have AVX-VNNI features now supported on both P-cores as well as E-cores. These backported AVX fallback paths are highly optimized and now available in the latest versions of optimized libraries. To use these new AVX features, developers simply need to recompile their software with the latest SDK. Developers should confirm that their software is enabled with those fallback paths and working as expected. Our 12th Generation Intel Core processors have enhanced support for a variety of instructions over previous generations of Intel CPUs.

Additionally, to make sure we have same ISA support on both P-cores and E-cores, we ported support for instructions listed below to E-cores (Previously, these were only supported on P-cores). For complete list of instructions please refer to [Intel® Architecture Instruction Set Extensions Programming Reference](#).

• ADOX/ADCX	• AVX^VAES (256 vex)	• F16C	• MKTIME
• AVX	• AVX2(and FMA)	• Fast-short rep-move (uArch perf, not ISA)	• PCCONFIG
• AVX^GFNI(128b and 256b vex)	• AVX-VNNI	• HRESET	• PKS (no new instructions)
• AVX^PCLMU LWDW(128b vex)	• BMI1	• INVPCID	• PKU
• AVX^PCLMU LWDW (256 vex)	• BMI2	• KEYLOCKER	• RAO-INT (fused off for hybrid)
• AVX^VAES (128b vex)	• CET	• LZCNT	• SERIALIZE

Table 1. List of new instructions added to support on E-cores for 12th Generation Intel Core processors

Additionally, there is available support for lightweight sleep instructions like the User Mode Wait or **UMWAIT** instruction and timed pause or **TPAUSE** instructions, through the 12th Generation Intel Core processor ISA. This support for these instructions is fully enabled through the Intel Thread Director, through the OS, and through threading libraries like oneTBB, OpenMP, Microsoft concurrency libraries. All of these libraries are already released and publicly available.

Optimized Libraries

As mentioned above, in developing for our performance hybrid architecture, we optimized its use with several libraries. We are listing some of them here as examples. We recommend using optimized libraries to take full advantage of the new performance.

For workloads and configurations visit www.Intel.com/PerformanceIndex. Results may vary.

Intel® oneAPI Threading Building Blocks (OneTBB)

OpenVINO® (Open Visual Inference and Neural network Optimization)

Microsoft Windows Machine Learning (WinML)

Conclusion

In this paper, we analyzed probable issues and have provided optimizations and fixes for those issues by assisting the OS, libraries, and frameworks to provide performant solutions. We believe these solutions will significantly reduce the burden on developers and assist them in getting the most performance out of the box when using our 12th Generation Intel Core processors. Additional tips and recommendations related to debugging will be provided in the next paper.

If you want to read more about Intel and what we are doing in the performance hybrid architecture space, please refer to the main site [here](#).

References

- [CPUID Information for Intel® Processors Identification Utility](#)
- [Processor affinity and binding - IBM Documentation](#)
- [Multiple-Processor Scheduling in Operating System - GeeksforGeeks](#)
- [Intel® Architecture Instruction Set Extensions Programming Reference](#)
- [Introducing Intel® Threading Building Blocks](#)
- [Introduction to Windows Machine Learning | Microsoft Docs](#)
- [SetProcessAffinityMask function \(winbase.h\) - Win32 apps | Microsoft Docs](#)
- [PROCESS_POWER_THROTTLING_STATE \(processthreadsapi.h\) - Win32 apps | Microsoft Docs](#)
- [SetThreadInformation function \(processthreadsapi.h\) - Win32 apps | Microsoft Docs](#)
- [SetProcessInformation function \(processthreadsapi.h\) - Win32 apps | Microsoft Docs](#)
- [GitHub - oneapi-src/oneTBB: oneAPI Threading Building Blocks \(oneTBB\)](#)
- [SetProcessAffinityMask function \(winbase.h\) - Win32 apps | Microsoft Docs](#)
- [OpenVINO®™ Toolkit Overview - OpenVINO®™ Toolkit \(OpenVINO® toolkit.org\)](#)
- [Microsoft Windows Machine Learning* \(WINML\) \(Intel®.com\)](#)
- [Version Helper functions - Win32 apps | Microsoft Docs](#)
- [Different Operating Systems – GeeksforGeeks](#)
- [Sysinfoapi.h header - Win32 apps | Microsoft Docs](#)
- [System Services - Win32 apps | Microsoft Docs](#)
- [GetLogicalProcessorInformation function \(sysinfoapi.h\) - Win32 apps | Microsoft Docs](#)
- [GetLogicalProcessorInformationEx function \(sysinfoapi.h\) - Win32 apps | Microsoft Docs](#)
- [SYSTEM_LOGICAL_PROCESSOR_INFORMATION \(winnt.h\) - Win32 apps | Microsoft Docs](#)
- [Processor Groups - Win32 apps | Microsoft Docs](#)

Disclaimer

Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.



Performance varies by use, configuration and other factors. Learn more at www.intel.com/PerformanceIndex.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Intel does not control or audit third-party benchmark data or the web sites referenced in this document. You should visit the referenced web site and confirm whether referenced data are accurate.

No product or component can be absolutely secure. Intel does not control or audit third-party data. You should consult other sources to evaluate accuracy.

Your costs and results may vary. Intel technologies may require enabled hardware, software or service activation.

© Intel Corporation.

Intel®, the Intel® logo, and other Intel marks are trademarks of Intel® Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.